

Tablekeeper

A reservation factory that keeps the guest promise.

Verified four-stage reservation system

Spec-derived checks

Reproduces from a fresh clone

THE PROBLEM

When a table closes mid-service, a booking must not be **silently moved** — the guest's time and accepted terms are the promise.

What the service guarantees

- Closure replans are **previewed by a manager**, never applied silently.
- Applying a plan is **atomic**: closure recorded and bookings moved together.
- Moved bookings keep their **time, owner, party size, and accepted terms**.

Why it is hard

- IANA timezones and DST make “the same clock time” non-trivial.
- Retries must be **idempotent** — a double submit must not double-book.
- Policies, revisions, and recurring series all interact with every change.

Search → book → **replan** → promise kept

Search & book

Availability grid, single or combined tables, timezone-correct slots.



Closure preview

Manager previews a deterministic repair plan for a closed table interval.



Atomic apply

Closure + moves commit together; affected bookings gain a reassigned entry.



Guest view holds

Availability and lookup reflect the new table — same time, same terms.

Booking surface

- Auth, search, create, amend, move, cancel, lookup
- Combinable table pairs for larger parties
- Idempotency-Key safe retries

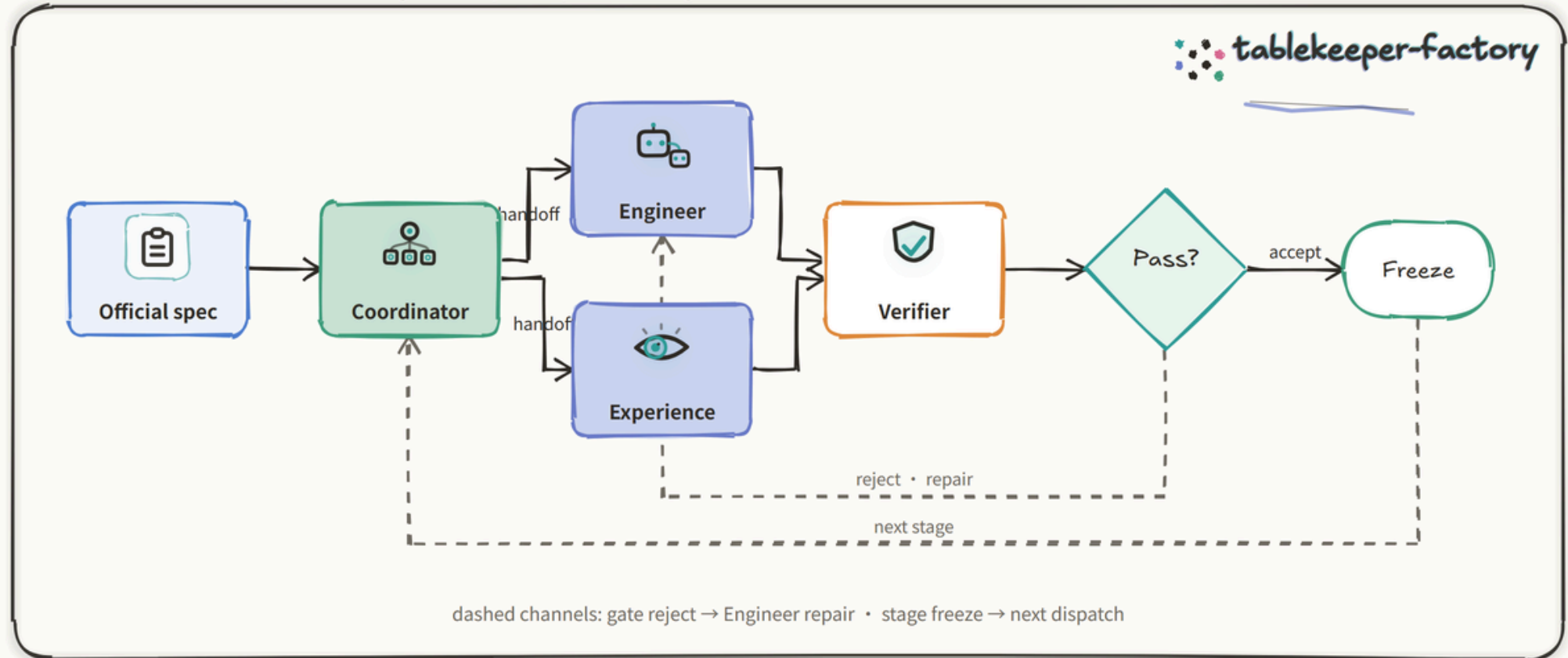
Management surface

- Effective-dated policies and revisions
- Recurring series with owner-only time amendments
- Reservation history and terms, end to end

The Dark

Factory

one dispatch • spec-derived gates • reject loops back to Engineer



Independent gates, reproduced

EVIDENCE	SOURCE	RESULT
Official harness, stage 1	final isolated sweep	120 / 120
Official harness, stage 2	final isolated sweep	25 / 25
Official harness, stage 3	final isolated sweep	7 / 7
Official harness, stage 4	final isolated sweep	6 / 6
Independent stage-4 verifier	combined API suite	862 / 862
Upgrade / hardened / UI legs	verifier suites	62 · 5 · 82 / all
Reproducibility	fresh public GitHub clone	full chain reproduced

Counts are checks executed in the published harnesses and verifier suites — not unique requirements, and not a guarantee of hidden judging tests.

What is proven — and what is not

Proven

- Four frozen stage releases, each independently buildable
- Published harness passes recorded per stage, incl. clean-clone run
- Committed mandates, handoffs, repair history, gate records

Not claimed

- **Not** zero-intervention: operator recovered runtimes, restarted seats, recorded gates
- Per-seat cost and stage wall-clock time are **unknown**
- No production deployment, users, or revenue; demo data is synthetic
- Published checks $\neq$ hidden-test success

TRY IT

Everything is in the repository

github.com/mukeshkbj/wed-dark-factory-tablekeeper

Run the product

- `stage-4/RUN.md` — one-command Docker build
- `docs/DEMO-RUNBOOK.md` — synthetic loopback demo
- MIT licensed · Python, no runtime dependencies

Inspect the factory

- `dispatch.md`, `mandates/`, `docs/handoffs/`
- `docs/gates/` — freeze records per stage
- `FACTORY.md` — full report incl. interventions